LAB: Stepper Motor

Date: 2026-10-28

Author/Partner: Haeun Yang

Github: https://github.com/haeunyang5/EC-STM32F4-22300442

Demo Video: demo video link

PDF version: LAB_Stepper_Motor_22300442_HaeunYang.pdf

Introduction

In **this lab**, a unipolar stepper motor (28BYJ-48) is driven by an STM32F411 through a ULN2003 driver. The coil excitation sequence is implemented as a Moore finite state machine (FSM) for both full-stepping and half-stepping. Using this FSM, the motor is rotated by a given number of steps at a given speed [rpm] in either direction, the maximum unloaded speed is measured, and an emergency stop is implemented with an external interrupt (EXTI).

Requirement

Hardware

- MCU
  - NUCLEO-F411RE
- Actuator/Sensor/Others:
  - Stepper Motor 28BYJ-48 (5V)
  - Motor Driver ULN2003
  - breadboard

Software

- VS Code (PlatformIO), CMSIS, EC_HAL library

Problem: Stepper Motor Control with FSM

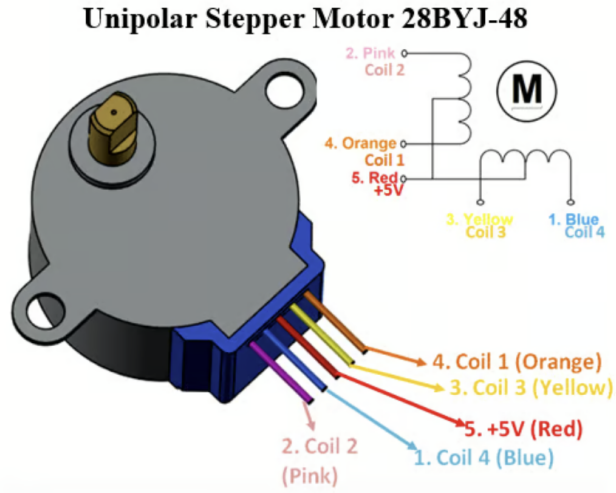
Problem Description

The stepper motor is controlled with the following procedure.

1. Find the number of steps per revolution in full-stepping.
2. Rotate the motor about 1 revolution at 1 rpm.
3. Rotate the motor in the opposite direction.
4. Increase the speed and find the maximum speed without load.
5. Stop the motor immediately when the user button (B1, PC13) is pressed, using EXTI (emergency stop).
6. Apply half-stepping and repeat 1–5.

Motor and Driver Specification

- **Driver:** ULN2003 Darlington transistor array (ULN2003 spec sheet)
- **Motor:** 28BYJ-48, 5 V unipolar, 4 phases. The internal motor has 32 full steps per revolution (11.25°/step) and a 1/64 gear reduction, so the output shaft needs 32 × 64 = **2048 full steps** (4096 half steps) per revolution.

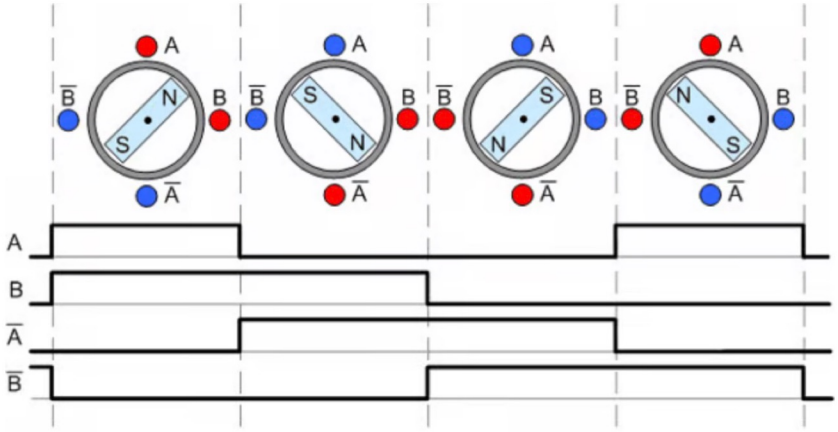


- Rated Voltage: 5V DC
- Number of Phases: 4
- Stride Angle: 5.625°/64
- Gear ratio: 1/32
- Pull in torque: 300 gf.cm
- Coil: Unipolar 5 lead coil

FSM design for Stepper Motor Sequence

State Table for Full-Stepping

- **Full-stepping sequence**



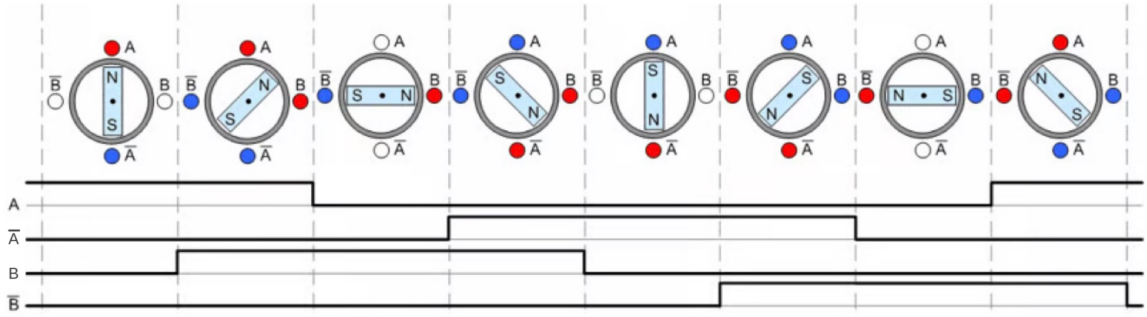
Phase	Port_Pin	Sequence			
		1	2	3	4
A	**PB_10**	H	L	L	H
B	**PB_4**	H	H	L	L
A′	**PB_5**	L	H	H	L
B′	**PB_3**	L	L	H	H

- **State Table**: Moore FSM. The input is the direction (DIR), and the output is the level of A, B, A', B'. DIR=0 follows the sequence S0→S1→S2→S3 and DIR=1 follows the reverse sequence. On the 28BYJ-48 used in this lab, DIR=0 rotated the output shaft counterclockwise, so `DIR_CCW = 0` and `DIR_CW = 1`.

State	Next State		Output
	DIR=0	DIR=1	(A B A' B')
S0	S1	S3	1 1 0 0
S1	S2	S0	0 1 1 0
S2	S3	S1	0 0 1 1
S3	S0	S2	1 0 0 1

State Table for Half-Stepping

- **Half-stepping sequence**

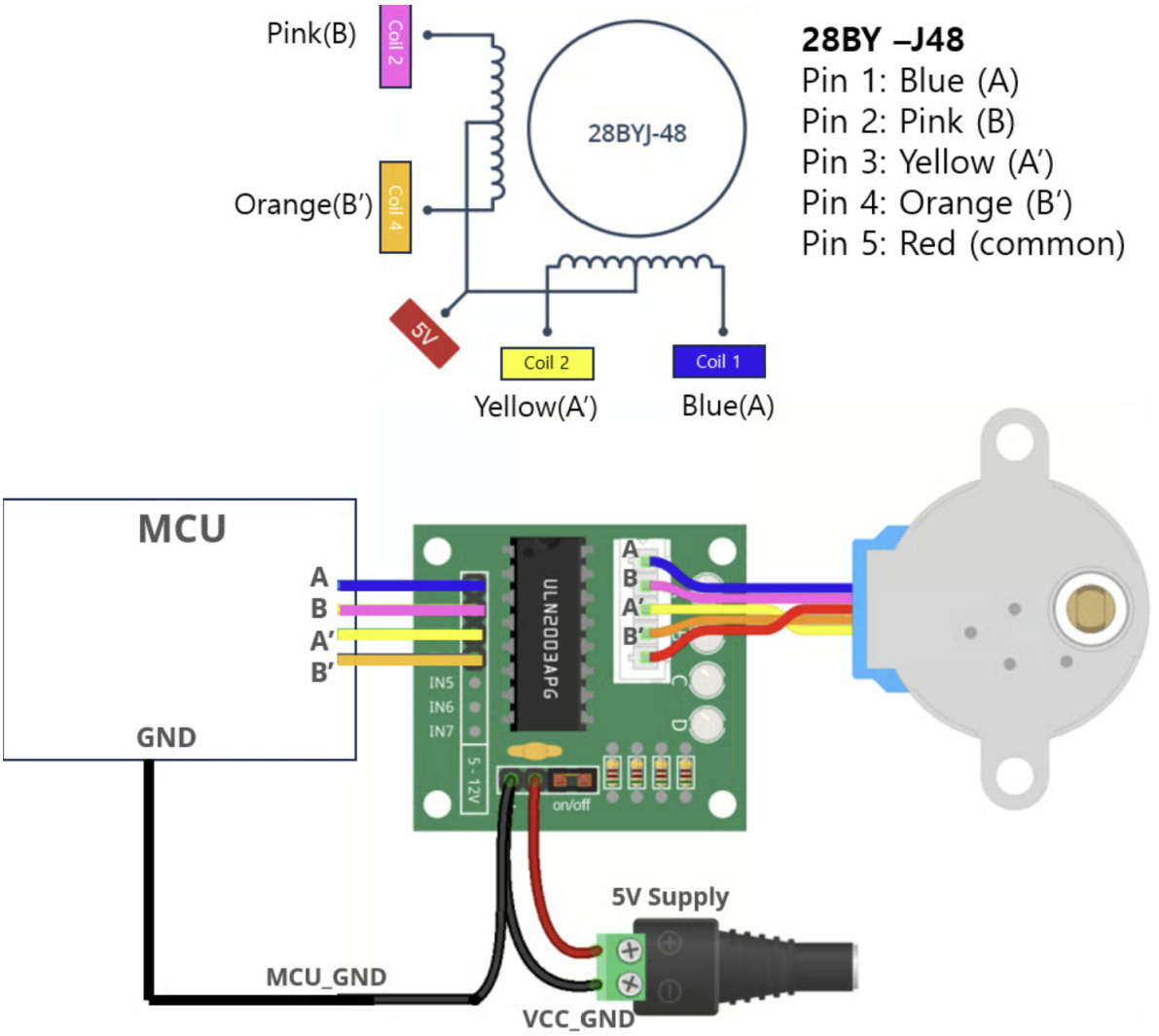

A
B
B
A
B
B
A
B
B

Configuration

Function	Pins	Configuration
Stepper output A, B, A', B'	PB10, PB4, PB5, PB3	DOUT, FAST speed
Button (B1, emergency stop)	PC13	DIN, EXTI falling edge, priority 0
System clock		PLL 84 MHz
SysTick		1 ms tick (`delay_ms()`)

Connection Diagram

The four outputs of the MCU (PB10, PB4, PB5, PB3) are connected to IN1–IN4 of the ULN2003 board, and the motor connector is plugged into the driver. The driver board is supplied with 5 V, and its GND is connected to the GND of the MCU.



Code

Function Definitions (ecStepper.h)

```
// Stepper Mode
#define HALF 0
#define FULL 1
#define DIR_CW 1
#define DIR_CCW 0

//State number
typedef enum {
    S0, S1, S2, S3, S4, S5, S6, S7
} StateNum;

// FULL stepping sequence  - FSM
typedef struct {
  uint32_t next[2];
  uint8_t out[4];
} StateStepper_t;

// Initialization of Stepper Motor
void stepper_init(uint32_t mode, PinName_t *pinStepper);

// Return step per revolution, always return 1 rpm
uint32_t stepper_stepPerRev(void) ;

// Converts Motor [rpm] to step delay in [msec]
void stepper_speed (uint32_t speedRPM);

// Run Stepper Motor for given steps and direction
void stepper_step(uint32_t steps, uint32_t direction) ;

// Stepper Motor Output for given state
void stepper_out(uint32_t state);

// Stop Stepper Motor
void stepper_stop(void);
```

FSM Tables (ecStepper.c)

The FSM is stored as a constant table. `next[0]` is the next state for DIR=0 (`DIR_CCW`) and `next[1]` is the next state for DIR=1 (`DIR_CW`). `out[]` is the output of the state (Moore FSM).

```
// Output: A B A' B'
static const StateStepper_t fsm_Full[4] = {
    {{S1,S3},{HIGH,HIGH,LOW,LOW}},
    {{S2,S0},{LOW,HIGH,HIGH,LOW}},
    {{S3,S1},{LOW,LOW,HIGH,HIGH}},
    {{S0,S2},{HIGH,LOW,LOW,HIGH}}
};

static const StateStepper_t fsm_Half[8] = {
    {{S1,S7},{1,0,0,0}},
    {{S2,S0},{1,1,0,0}},
    {{S3,S1},{0,1,0,0}},
    {{S4,S2},{0,1,1,0}},
```

```
    {{S5,S3},{0,0,1,0}},
    {{S6,S4},{0,0,1,1}},
    {{S7,S5},{0,0,0,1}},
    {{S0,S6},{1,0,0,1}}
};
```

A pointer `fsm` points to `fsm_Full` or `fsm_Half` depending on the mode chosen in `stepper_init()`, so the same `stepper_step()` and `stepper_out()` functions are used for both modes.

---

stepper_init()

Selects the FSM table and the number of steps per revolution according to the drive mode (Full/Half), and initializes the four output pins (A, B, A', B') as GPIO outputs.

```
void stepper_init(uint32_t mode, PinName_t *pinStepper)
```

Parameters

- `mode` : Drive mode. `FULL` or `HALF`
- `pinStepper` : Array of pins connected in the order A, B, A', B'

```
void stepper_init(uint32_t mode, PinName_t *pinStepper){
    if (mode == HALF) {
        fsm = fsm_Half;
        STEP_PER_REV = 4096 ;
    }
    else{
        fsm = fsm_Full;
        STEP_PER_REV = 2048 ;
    }

    for (int i=0; i<4; i++){
        stepperPins[i] = pinStepper[i];
        GPIO_init(stepperPins[i], OUTPUT) ;
        GPIO_ospeed(stepperPins[i], FAST_SPEED) ;
    }
}
```

Example code

```
PinName_t pinStepper[4] = {PB_10, PB_4, PB_5, PB_3};   // A, B, A', B'
stepper_init(FULL, pinStepper);
```

---

stepper_stepPerRev()

Returns the number of steps required for one revolution of the output shaft in the current mode (Full: 2048, Half: 4096).

```
uint32_t stepper_stepPerRev(void)
```

Example code

```
uint32_t rev   = 1;
uint32_t steps = stepper_stepPerRev() * rev;
```

---

stepper_speed()

Converts the motor speed [rpm] into the delay time per step [ms] and sets it.

```
void stepper_speed(uint32_t speedRPM)
```

Parameters

- `speedRPM`: Target rotation speed [rpm]

```
void stepper_speed (uint32_t speedRPM){
    _step_delay = 60*1000/(STEP_PER_REV*speedRPM) ;   // [msec]
}
```

Example code

```
stepper_speed(1);   // 1 rpm -> 29 ms/step (full), 14 ms/step (half)
```

---

stepper_step()

Rotates the motor by the given number of steps in the given direction. At each step, the next state is determined from the present state and the direction, the output of that state is applied, and then the function waits for `_step_delay`. Since the loop condition reads the global `_steps`, the loop ends immediately when `stepper_stop()` sets `_steps` to 0 in an interrupt.

```
void stepper_step(uint32_t steps, uint32_t direction)
```

Parameters

- `steps`: Number of steps to rotate
- `direction`: Rotation direction. `DIR_CW` or `DIR_CCW`

```
void stepper_step(uint32_t steps, uint32_t direction){
    if (fsm == 0) return ;           // exit if stepper_init() was not called
    _steps = steps ;

    for (uint32_t i = 0; i < _steps; i++){
        // Update present state
        if (direction == DIR_CCW) state = fsm[state].next[0] ;
        else state = fsm[state].next[1] ;

        stepper_out(state) ;         // output of present state
        delay_ms(_step_delay) ;      // delay for motor speed
    }
}
```

Example code

```
stepper_step(stepper_stepPerRev(), DIR_CW);   // one revolution, clockwise
```

---

stepper_out()

Reads the output values of the given state from the FSM table and writes them to the A, B, A', B' pins.

```
void stepper_out(uint32_t state)
```

Parameters

- `state` : FSM state number (Full: S0–S3, Half: S0–S7)

```
void stepper_out(uint32_t state){
    for (int i = 0; i < 4; i++) {
        GPIO_write(stepperPins[i], fsm[state].out[i]);
    }
}
```

Example code

```
stepper_out(S0);   // Full: A=1, B=1, A'=0, B'=0
```

---

stepper_stop()

Sets the remaining step count to 0 so that the rotation loop in `stepper_step()` ends, and sets all coil outputs to LOW to stop the motor.

```
void stepper_stop(void)
```

```
void stepper_stop(void) {
    _steps = 0;
    for (int i=0; i<4; i++) GPIO_write(stepperPins[i], 0) ;
}
```

Example code

```
stepper_stop();   // called in EXTI15_10_IRQHandler()
```

Main Code (LAB_Stepper_Motor.c)

```
#include "stm32f411xe.h"
#include "ecSTM32F4v2.h"

// ( A, B,  AN,  BN)
PinName_t pinStepper[4]    = {PB_10,PB_4,PB_5,PB_3};
#define BUTTON_PIN PC_13

// Initialiization
```

```
void setup(void){
    RCC_PLL_init();                                 // System Clock = 84MHz
    SysTick_init();                                 // Systick init

    GPIO_init(BUTTON_PIN, INPUT);                   // GPIOC pin13
initialization
    EXTI_init(BUTTON_PIN, FALL, 0);                 // External Interrupt
Setting

    stepper_init(HALF, pinStepper);                 // Stepper GPIO pin
initialization
    stepper_speed(2);                               //  set stepper motor speed
}


int main(void) {
    // Initialiization --------------------------------------------------------
    setup();
    uint32_t rev=1;
    uint32_t steps=stepper_stepPerRev()*rev;

    // Inifinite Loop ---------------------------------------------------------
    while(1){
        stepper_step(steps, DIR_CW);  // (Step : 2048, Direction : 0 or 1, Mode
: FULL or HALF)
        delay_ms(2000);
        stepper_step(steps, DIR_CCW);  // (Step : 2048, Direction : 0 or 1, Mode
: FULL or HALF)
        delay_ms(2000);
    }
}



void EXTI15_10_IRQHandler(void) {
    if (is_pending_EXTI(BUTTON_PIN)) {
        stepper_stop();
        clear_pending_EXTI(BUTTON_PIN); // cleared by writing '1'
    }
}
```

- The mode argument of `stepper_init()` (`FULL` / `HALF`) is changed for each procedure.
- The main loop rotates the motor one revolution CW, waits 2 s, rotates one revolution CCW, and waits 2 s.

Results

Procedure	Full-stepping	Half-stepping
Steps per revolution	2048	4096
Step delay at 1 rpm (`stepper_speed(1)`)	29 ms	14 ms
Time for 1 revolution at 1 rpm	58 s	56 s
Maximum speed	14 rpm	14 rpm

Analysis. In full-stepping, one revolution took 2048 steps as calculated from the motor specification, and the rotation direction was reversed by selecting the other column of `next[]` in the FSM table. At 1 rpm, one revolution took 58 s instead of 60 s because of the integer truncation of the step delay and the resolution of `delay_ms()` (see Troubleshooting).

In half-stepping, twice as many steps are needed for one revolution, so the step delay is halved for the same rpm. One revolution at 1 rpm took 56 s. The step delay is truncated from 14.65 ms to 14 ms, and `delay_ms()` is about 0.5 ms shorter on average, so one revolution takes about 4096 × 13.5 ms ≈ 55.3 s, which agrees with the measurement.

The maximum speed was 14 rpm in both modes. In full-stepping, the motor rotated with a 2 ms step delay but did not start with 1 ms, because the torque of the stepper motor decreases at a high step rate. In half-stepping, 14 rpm corresponds to a step delay of about 1 ms (60000 / (4096 × 14) ≈ 1.05 ms → 1 ms), so the motor ran with a shorter delay than in full-stepping, because each half step is a smaller movement of the rotor.



Demo video: [demo video link]

Discussion

1. Find out the trapezoid-shape velocity profile for a stepper motor. When is this profile necessary?

A trapezoidal velocity profile consists of three phases: constant acceleration, constant speed, and constant deceleration (Atmel Corporation, 2006). Instead of starting at the target speed immediately, the step delay is gradually decreased during acceleration and increased during deceleration.

This profile is necessary because the torque of a stepper motor decreases as speed increases. The speed at which the motor can start directly from rest (pull-in) is lower than the speed it can maintain once running (pull-out) (Condit & Jones, 2004). Therefore, when the target speed is high or the load inertia is large, an abrupt start causes missed steps or stalling, and an abrupt stop causes overshoot. In this experiment, the motor did not start when the step delay was reduced from 2 ms to 1 ms, which an acceleration ramp could overcome.

2. How would you change the code more efficiently for micro-stepping control? You don't have to code this but need to explain your strategy.

Micro-stepping drives the two coils with sine and cosine currents instead of on/off signals, so the rotor can stop between full steps (Chang & Tran, 2025). The strategy is as follows:

- **Output table:** Replace the 0/1 output table with a sine lookup table of PWM duty cycles. Only a quarter of the sine wave needs to be stored because of its symmetry.
- **Next state:** Calculate the next state as `index = (index ± 1) % N` instead of storing a `next[]` table, since the number of states becomes large.
- **Output method:** Drive the coils with timer PWM outputs instead of `GPIO_write()`. All four pins used (PB10, PB4, PB5, PB3) support timer PWM (STMicroelectronics, 2024).
- **Timing:** Replace `delay_ms()` with a timer interrupt, because micro-steps require step periods shorter than 1 ms.

3. There are other types of Stepper Motor Drivers that are simple to use, such as you only give one pulse signal and direction, instead of giving 4 pulse signals. Such examples are A4988, DRV 8834, and TB6600 drivers. Compare these motor drivers with ULN2003 in terms of operating method.

The ULN2003 is only a Darlington transistor array without any sequencing logic (Texas Instruments, 2025). The microcontroller must generate the phase sequence and control four pins at every step, as done by the FSM in this experiment.

In contrast, the A4988, DRV8834, and TB6600 have a built-in translator. The microcontroller only sends one pulse to the STEP pin to move one step and sets the direction with the DIR pin (Allegro MicroSystems, 2022; Texas Instruments, 2015; Toshiba Corporation, 2014). These drivers also regulate the coil current and support micro-stepping in hardware, which reduces the CPU load.

Item	ULN2003	A4988 / DRV8834 / TB6600
Control signals	4 phase signals	STEP + DIR
Step sequence	MCU software (FSM)	Built-in translator
Current control	None	PWM current regulation
Micro-stepping	Not supported	Supported (up to 1/16 or 1/32)
MCU load	High	Low

Reference

Allegro MicroSystems. (2022). *A4988: DMOS microstepping driver with translator and overcurrent protection* (Datasheet 4988-DS, Rev. 8). https://www.allegromicro.com/-/media/files/datasheets/a4988-datasheet.pdf

Atmel Corporation. (2006). *AVR446: Linear speed control of stepper motor* (Application Note 8017A–AVR–06/06). https://www.microchip.com/en-us/application-notes/an8017

Austin, D. (2004, December 30). Generate stepper-motor speed profiles in real time. *Embedded.com*. https://www.embedded.com/?p=4006438

Chang, C., & Tran, T. (2025). Mastering precision: Understanding microstepping in motion control. *Analog Dialogue, 59*(2). https://www.analog.com/en/resources/analog-dialogue/articles/mastering-precision-understanding-microstepping.html

Condit, R., & Jones, D. W. (2004). *Stepping motors fundamentals* (Application Note AN907, DS00907A). Microchip Technology Inc. https://www.microchip.com/en-us/application-notes/an907

STMicroelectronics. (2024). *STM32F411xC STM32F411xE: Arm Cortex-M4 32-bit MCU+FPU* (Datasheet DS10314, Rev. 8). https://www.st.com/resource/en/datasheet/stm32f411re.pdf

Texas Instruments. (2015). *DRV8834 dual-bridge stepper or DC motor driver* (Datasheet SLVSB19D). https://www.ti.com/lit/ds/symlink/drv8834.pdf

Texas Instruments. (2025). *ULN200x, ULQ200x high-voltage, high-current Darlington transistor arrays* (Datasheet SLRS027T). https://www.ti.com/lit/ds/symlink/uln2003a.pdf

Toshiba Corporation. (2014). *TB6600HG: PWM chopper-type bipolar stepping motor driver IC* [Datasheet]. https://www.makerguides.com/wp-content/uploads/2019/10/TB6600HG-Datasheet.pdf

Trouble shooting

When the motor was driven at 1 rpm, one revolution was expected to take 60 s, but the measured time was 58 s. Two sources of timing error were identified.

First, the step delay is calculated in milliseconds using integer division, so the fractional part is truncated:

$$\text{delay} = \frac{60 \times 1000}{2048 \times 1} = 29.30\ \text{ms} \ \rightarrow \ 29\ \text{ms}$$

This alone shortens one revolution to

$$2048 \times 29\ \text{ms} \approx 59.4\ \text{s}$$

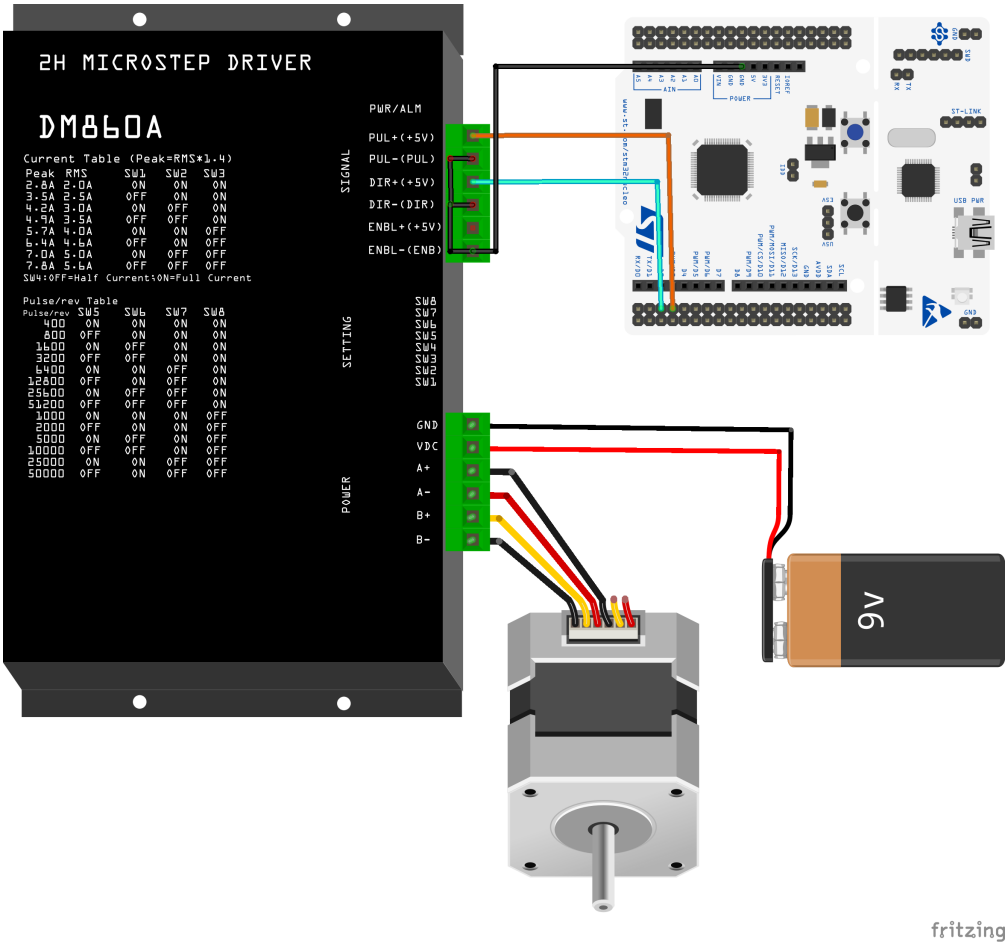
Second, `delay_ms()` counts 1 ms SysTick ticks. Because the function is called at an arbitrary point within a tick, the first tick arrives earlier than 1 ms, and the actual delay lies between (n-1) and n ms, which is about 0.5 ms shorter on average. Since this error accumulates over 2048 steps, one revolution becomes approximately

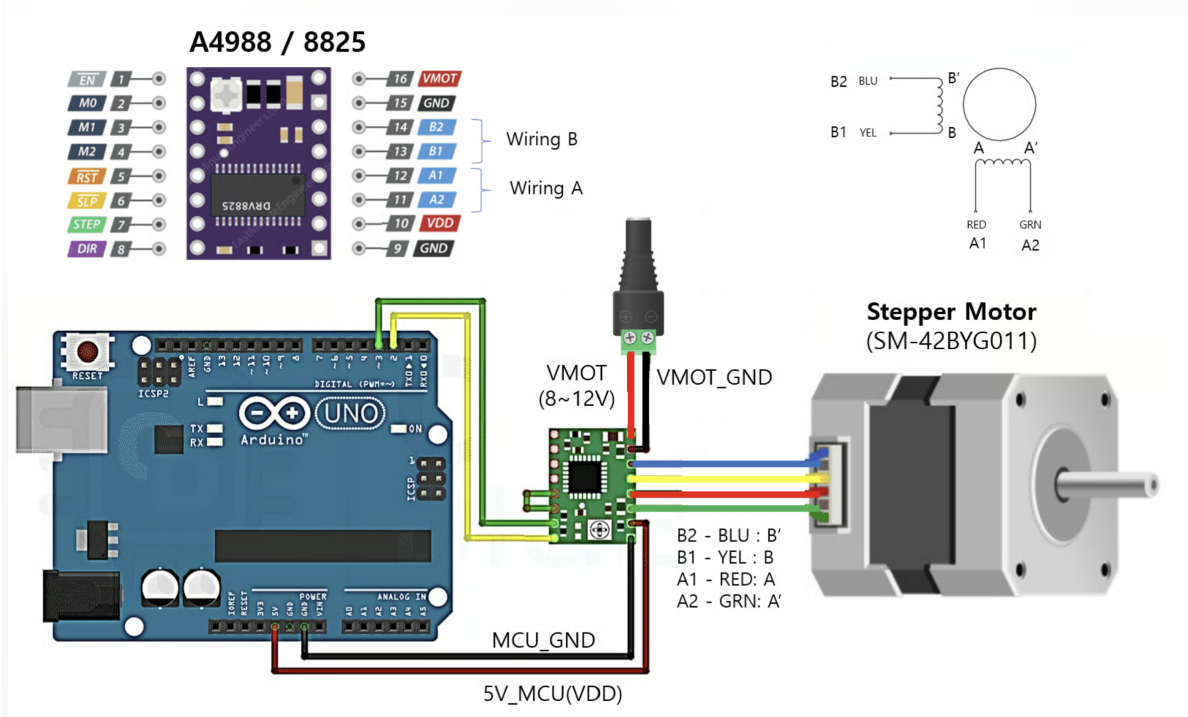
$$2048 \times 28.5\ \text{ms} \approx 58.4\ \text{s}$$

which agrees well with the measured value.

Both errors can be reduced by calculating the step delay in microseconds and generating it with a microsecond-resolution timer, such as `delay_us()` or a hardware timer interrupt.

Tutorial: Stepper Motor Driver (A4988)





Tutorial code

```
#include "stm32f411xe.h"
#include "ecRCC2.h"
#include "ecGPIO2.h"
#include "ecSysTick2.h"

#define MOTOR_DIR PB_3
#define MOTOR_PULSE PA_10

void setup(void)
{
    RCC_PLL_init();
    SysTick_init();
    GPIO_init(MOTOR_DIR, OUTPUT);
    GPIO_init(MOTOR_PULSE, OUTPUT);
}


int main(void) {
    setup();
    // Set Dir high (CW)
    GPIO_write(MOTOR_DIR, HIGH);
    while(1){
          for(x = 0; x < 1000; x++)
          {
            GPIO_write(MOTOR_PULSE, HIGH);
            delay_ms(10);
            GPIO_write(MOTOR_PULSE, LOW);
            delay_ms(10);
          }
        delay_ms(1000);
    }
}
```